Library Management System

Software Test Design Document

Tishko

June 13, 2025

1 Authentication Controller (authController)

1.1 Overview

The Auth Controller is responsible for user authentication, including registration (by a librarian), login, and logout. These tests ensure that access control and user session management are functioning correctly, as specified in the SRS.

Table 1: Test Cases for authController

Test Case ID	**Related Requirement (SRS)**	**Test Scenario**	**Test Steps**	**Expected Result**
TC-AUTH-001	3.1.1 User Registration , UC-004	Successful user registration	1. Mock `User.findOne` to resolve as null. 2. Mock `User.create` to resolve with a new user object. 3. Call `registerUser` with valid new user data.	1. Response status is 201. 2. Response JSON indicates successful registration.
TC-AUTH-002	UC-004 (A1: Email Already Exists)	Attempt to register a user that already exists	1. Mock `User.findOne` to resolve with an existing user object. 2. Call `registerUser` with data of an existing user.	1. Response status is 400. 2. Response JSON indicates that the username or email already exists.
TC-AUTH-003	3.1.2 User Login , UC-001	Successful user login with valid credentials	1. Mock `passport.authenticate` to succeed and return a user object. 2. Call `loginUser` middleware with correct username and password.	1. `req.logIn` is called. 2. Response status is 200. 3. Response JSON indicates successful login and includes user data.

Continued on next page

Table 1 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-AUTH-004	UC-001 (A1: Invalid Credentials)	Attempt to login with invalid credentials	1. Mock `passport.authenticate` to fail (returns false). 2. Call `loginUser` middleware with an incorrect username or password.	1. Response status is 401. 2. Response JSON indicates incorrect credentials.
TC-AUTH-005	3.1.3 User Logout	Successful user logout	1. Mock `req.logout`. 2. Call `logoutUser`.	1. `req.logout` is called. 2. Response status is 200. 3. Response JSON indicates successful logout.

2 Author Controller (authorController)

2.1 Overview

The Author Controller manages CRUD operations for author entities. These tests validate the creation and retrieval of authors.

Table 2: Test Cases for authorController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-ATHR-001	3.4 Author Management (POST /api/author/add)	Successfully create a new author	1. Provide author details in `req.body`. 2. Mock `Author.create` to resolve with the new author data. 3. Call `createAuthor`.	1. Response status is 201. 2. Response JSON contains the newly created author.
TC-ATHR-002	3.4 Author Management (GET /api/author/getAll)	Successfully retrieve all authors	1. Mock `Author.find` to resolve with an array of author objects. 2. Call `getAllAuthors`.	1. Response status is 200. 2. Response JSON contains the array of all authors.

3 Book Controller (bookController)

3.1 Overview

The Book Controller handles all CRUD operations for book entities. The tests ensure that books can be created, retrieved (all and by ID), updated, and deleted as per the requirements.

Table 3: Test Cases for bookController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BOOK-001	3.3 Book Management (POST /api/book/add) , UC-002	Successfully create a new book	1. Provide book details in `req.body`. 2. Mock `Book.create` to resolve successfully. 3. Call `createBook`.	1. Response status is 201. 2. Response JSON contains the new book.

Continued on next page

Table 3 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BOOK-002	UC-002 (A2: API Error)	Fail to create a book due to a database error	1. Mock `Book.create` to reject with an error. 2. Call `createBook`.	1. Response status is 500. 2. Response JSON contains an error message.
TC-BOOK-003	3.3 Book Management (GET /api/book/getAll)	Successfully retrieve all books	1. Mock `Book.find().populate()` to resolve with an array of books. 2. Call `getAllBooks`.	1. Response status is 200. 2. Response JSON contains the array of books.
TC-BOOK-004	3.3 Book Management (GET /api/book/get/:id)	Successfully retrieve a single book by its ID	1. Provide a book ID in `req.params`. 2. Mock `Book.findById().populate()` to resolve with a book object. 3. Call `getBookById`.	1. Response status is 200. 2. Response JSON contains the specific book.
TC-BOOK-005	3.3 Book Management (GET /api/book/get/:id)	Attempt to retrieve a book with a non-existent ID	1. Provide a non-existent book ID in `req.params`. 2. Mock `Book.findById().populate()` to resolve as null. 3. Call `getBookById`.	1. Response status is 404. 2. Response JSON contains a 'Book not found' message.
TC-BOOK-006	3.3 Book Management (PUT /api/book/update/:id)	Successfully update an existing book	1. Provide a book ID and update data. 2. Mock `Book.findByIdAndUpdate` to resolve with the updated book. 3. Call `updateBook`.	1. Response status is 200. 2. Response JSON contains the updated book data.
TC-BOOK-007	3.3 Book Management (DELETE /api/book/delete/:id)	Successfully delete a book	1. Provide a book ID to be deleted. 2. Mock `Book.findByIdAndDelete` to resolve successfully. 3. Call `deleteBook`.	1. Response status is 200. 2. Response JSON contains a 'Book removed' message.

4 Borrowal Controller (borrowalController)

4.1 Overview

This controller manages borrowal records. Tests cover creating a borrowal (and updating book availability), updating a borrowal's status, and retrieving borrowals based on user roles (Librarian vs. Member).

Table 4: Test Cases for borrowalController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BORR-001	3.6 Borrowal Management , UC-003	Create a borrowal for an available book	1. Mock `Book.findById` to return an available book. 2. Mock `Borrowal.create` to succeed. 3. Call `createBorrowal`.	1. Book's availability is set to false. 2. Response status is 201. 3. Response JSON contains the new borrowal record.

Continued on next page

Table 4 – continued from previous page

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-BORR-002	UC-003 (A1: Book Not Available)	Attempt to create a borrowal for an unavailable book	1. Mock `Book.findById` to return an unavailable book. 2. Call `createBorrowal`.	1. Response status is 400. 2. Response JSON has a "Book is not available" message.
TC-BORR-003	3.6 Borrowal Management (PUT /api/borrowal/update/:id)	Update borrowal status to 'Returned'	1. Mock `Borrowal.findById` to find a borrowal. 2. Mock `Book.findByIdAndUpdate` to succeed. 3. Call `updateBorrowal` with status 'Returned'.	1. Book's availability is set to true. 2. Response status is 200. 3. Response JSON contains the updated borrowal.
TC-BORR-004	3.6 Borrowal Management (GET /api/borrowal/getAll)	Get all borrowals as a Librarian	1. Set `req.user.role` to 'Librarian'. 2. Mock `Borrowal.find` to return all borrowals. 3. Call `getBorrowals`.	1. `Borrowal.find` is called with an empty filter. 2. Response contains all borrowal records.
TC-BORR-005	3.6 Borrowal Management , UC-005	Get own borrowals as a Member	1. Set `req.user.role` to 'Member'. 2. Mock `Borrowal.find` to return member-specific borrowals. 3. Call `getBorrowals`.	1. `Borrowal.find` is called with a filter for the member's ID. 2. Response contains only the member's borrowals.

5 Genre Controller (genreController)

5.1 Overview

The Genre Controller manages CRUD operations for genre entities. These tests validate the creation and retrieval of genres.

Table 5: Test Cases for genreController

Test Case ID	Related Requirement (SRS)	Test Scenario	Test Steps	Expected Result
TC-GENR-001	3.5 Genre Management (POST /api/genre/add)	Successfully create a new genre	1. Provide genre details in `req.body`. 2. Mock `Genre.create` to resolve with the new genre data. 3. Call `createGenre`.	1. Response status is 201. 2. Response JSON contains the newly created genre.
TC-GENR-002	3.5 Genre Management (GET /api/genre/getAll)	Successfully retrieve all genres	1. Mock `Genre.find` to resolve with an array of genre objects. 2. Call `getAllGenres`.	1. Response status is 200. 2. Response JSON contains the array of all genres.

6 Review Controller (reviewController)

6.1 Overview

The Review Controller is responsible for managing book reviews. The tests cover the creation of reviews and fetching all reviews for a specific book.

Table 6: Test Cases for reviewController

Test Case ID	**Related Requirement (SRS)**	**Test Scenario**	**Test Steps**	**Expected Result**
TC-REV-001	3.8 Review Management (POST /api/review/add)	Successfully create a review for a book	1. Provide review details (bookId, rating, comment) in `req.body`. 2. Mock `Review.create` to resolve with the new review object. 3. Call `createReview`.	1. Response status is 201. 2. Response JSON contains the newly created review.
TC-REV-002	3.8 Review Management	Get all reviews for a specific book	1. Provide a book ID in `req.params.bookId`. 2. Mock `Review.find().populate()` to resolve with an array of reviews. 3. Call `getReviewsForBook`.	1. `Review.find` is called with a filter for the book ID. 2. Response status is 200. 3. Response JSON contains the array of reviews for that book.